

EECS 498 · AASE · FALL 2026

How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER · SEP 14–15, 2026

Schedule

1. Aider deep dive
2. Hackathon 1
3. The next assignment
4. Office hours

The harness

Aider is a *harness*: a program that stands between you and a model.

The harness	The model
Picks the text the model sees	Reads it
Decides what a reply may do	Writes a reply
Edits files, then commits	Nothing

Take one apart now, build one later.

What the model never touches

- Your disk
- Your shell
- Your last session

Text in, text out. The harness does the rest.

What writes, and what does not

Arranges context

`/add`

`/read-only`

`/drop`

`/tokens`

`/ask`

`/diff`

Changes your files

`/code`

`/undo`

Bare text means `/code`. The default writes.

Who writes the prompt

THERE IS NO CONVERSATION, ONLY AN ASSEMBLED BUFFER

Aider's first-person messages

"I have *added these files to the chat* so you can go ahead and edit them."

"I committed the changes with git hash 4f2a1c & commit msg: add search"

"I edited the files myself."

"I didn't see any properly formatted edits in your reply?!"

Aider's own strings, sent to the model as you.

Both halves of the transcript

Written by	Appears in the prompt as
Aider	Your file listings and status lines
Aider	The model agreeing to them
Aider	The format reminder, glued to your message
You	The sentence you actually typed

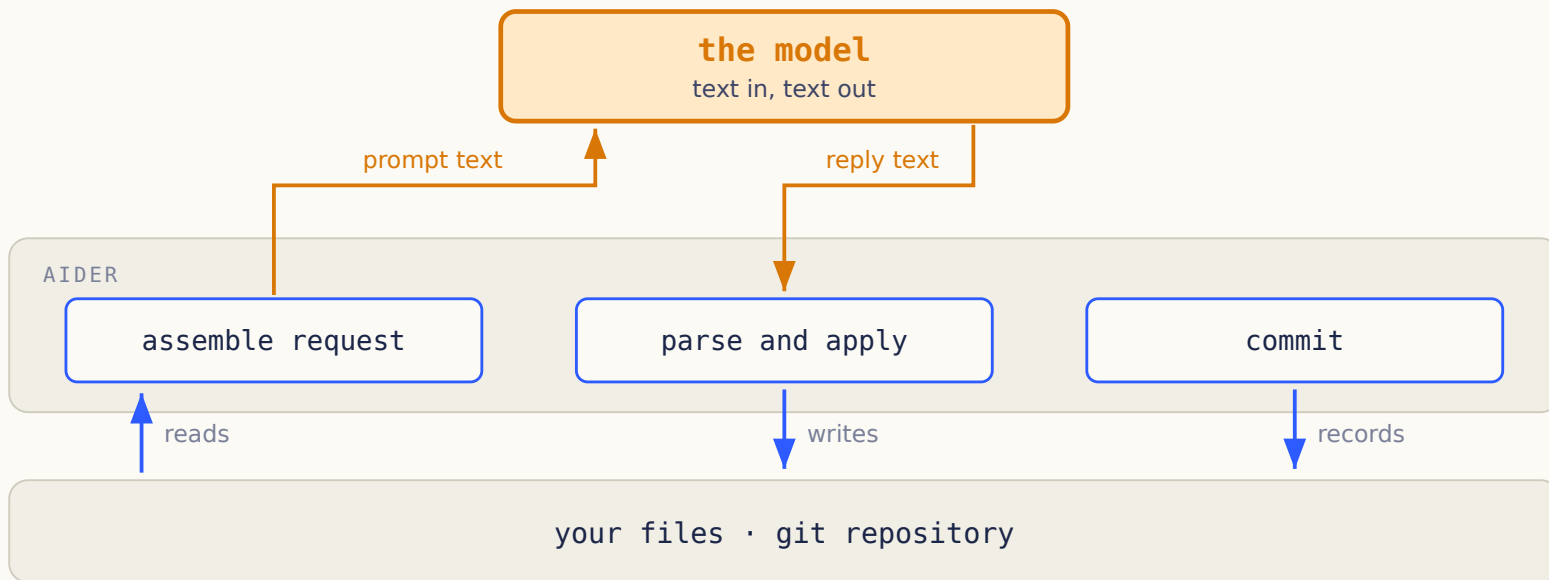
One author. Two voices.

Context

WHAT THE HARNESS CHOOSES TO SEND

WHO DOES WHAT

Aider, the model, and the repo



The model touches no file.

What goes into the request

Fixed each turn	Instructions, added files, repo map
Grows each turn	Session history, your message

Context is a selection. A file you never added is not in it.

Context as a budget

Competing for one window

Instructions and examples	Paid every turn
Session history	Grows until it is summarised
Added files	The ones you chose
Repo map	Whatever is left over

A harness is a policy for spending it.

The repo map

```
package/store.py:
:
|class Store:
|    def add(self, title, tags=()) -> Task:
|    def remove(self, task_id: int) -> bool:
:
|    def search(self, text, tag=None) -> list[Task]:
:

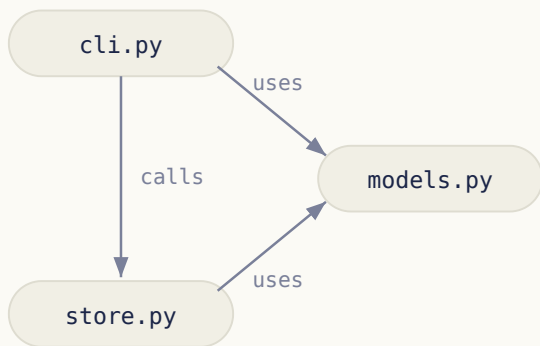
package/models.py:
|@dataclass
|class Task:
:
```

Signatures only. `:` is what was cut.

Files you never added.

Where the repo map comes from

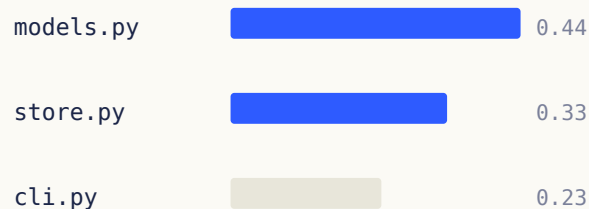
1. parse every file for what it defines and what it uses



models.py is used by both, so it wins

PageRank, rendered until the budget ends.

2. rank by what points at them



budget ends here

cli.py is left out entirely

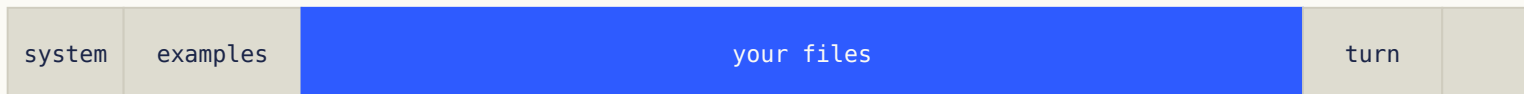
COST

What the map costs

repo map on



repo map off



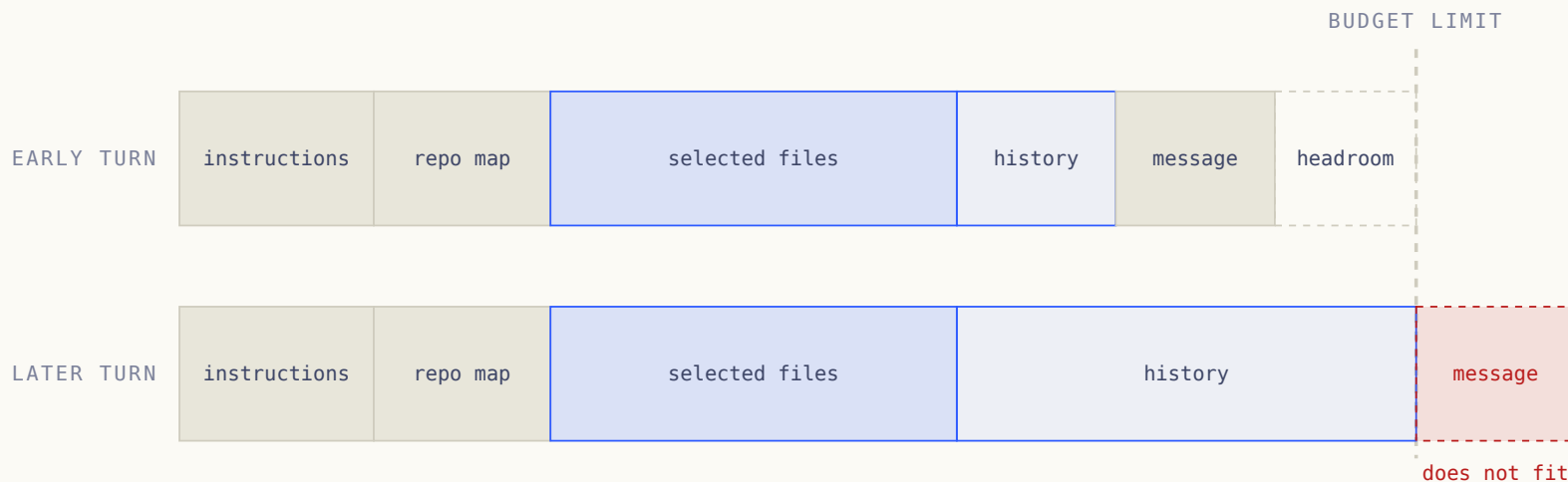
0

the whole context window

Same window either way.

On a small model the map can outweigh the files.

One shared token budget



One budget: history, map, and files.

What a turn costs

WHY THE SAME TEXT IS SENT AGAIN AND AGAIN

The prefix

System prompt	1
Examples	2
Read-only files	3
Repo map	4
Session history	5
Added files	6
Your message	7

Rebuilt in this order, from scratch, every turn.

Reusing a prefix

A server that has already processed a prefix does not process it again.

Unchanged prefix	Reused
First changed token	Everything after it is recomputed

Reuse is a prefix property, not a memory.

What goes cold

- Switching between ask and code
- Adding or dropping a file
- Naming a file in your message

The first changed token invalidates the rest.

The volatility ordering

Rarely changes	System prompt, examples
Changes on request	Read-only files, repo map
Changes on every edit	Added file contents

So an edited file is the cheapest thing to re-send.

THE REPLY

Edit-shaped replies

```
path/to/file.py
<<<<<<< SEARCH
the lines exactly as they appear on disk
=====
the lines that replace them
>>>>>>> REPLACE
```

One block, one file. Nothing forces the set to be complete.

An action through a text channel

The model returns

Prose

Prose

Prose

The harness needs

A file path

An exact region

The replacement text

Search blocks, tool-call JSON, tagged output. One problem.

Errors written for the model

Aider answers the model, not you.

- The failed block, quoted back
- Lines from the file that nearly match
- A note if the work already exists

An error message is a prompt.

How a file update arrives

Each added file is read from disk and sent whole, every turn.

Never sent	A diff of what changed
Never kept	The previous copy
Also sent	A line saying it committed

Edits made in your own editor arrive the same silent way.

STALE GROUND

What *it* remembers

The file block is rebuilt. The model's own replies are not.

It can see the lines it chose to change, and nothing else.

The silent undo

After an undo

The file	Quietly back to the old text
The model	Told nothing at all
The history	Still holds the edit it proposed

Its best evidence says the change was made.

THURSDAY SEPTEMBER 24

Hackathon 1

- Two hours, your own project
- Prompt revealed in the room
- Submit before you leave
- Graded separately from the build

The hackathon workspace

- Browser VS Code, already set up
- Aider pointed at the course endpoint
- Your repository already cloned
- A Python reference pack, offline

About 690,000 tokens of documentation. Your budget is 8,000.

Weight in the Apply grade

Share of the Apply grade

Hackathon 1	A half
The pair-programmer build	A quarter
The practice lessons	A quarter

Bring a runnable increment.

The pair- programmer

WHAT STAGE 1 ASKS FOR

A harness from scratch

Everything in the last hour, written by you.

It holds	A set of files you chose
It asks	A model endpoint you configure
It writes	Only after you approve

Aider without Aider. Same job, your code.

Due **Tuesday October 6, 11:59 PM.**

Specification before implementation

You write the requirements before you write the code, and the commit history has to show it.

System design	Once, before implementation
Increment specs	Slices you choose, each spec first

No specification is supplied.

The design artifacts

Artifact	Covers
Your requirements	Config, run contract, seven features
Increment specs	Ordered, testable, context scoped
Three diagrams	Components, sequence, edit lifecycle

Specification and design

50 of the 100 build points.

Criterion	Points
Requirements and acceptance criteria	10
Architecture and interface design	10
Increment specs	10
The three diagrams	4 / 3 / 3
Specs before code	5
What changed and why	5

Implementation and tests

50 of the 100 build points.

Criterion	Points
Functional correctness	25
Custom test cases	9
Failure-path tests	8
End-to-end runs and live evidence	5
Operating instructions	3

`RUBRIC.md` carries every row.

WHAT YOU KEEP

The evidence trail

Commits	Specs before the code they describe
Sessions	Which context, which model, what happened
Reconciliation	Where the design changed, and why

Model rules for specifications

- Drafting, critique, diagrams, code, tests
- 9B recommended, 4B permitted
- Secondary models recorded too
- Any compatible host allowed

A different provider is not a different model.

Why this tool

WHAT IT SHOWS YOU THAT A BETTER ONE HIDES

The Ledger

What it gives you

You choose the context

Every decision is visible

One loop you can follow

Nothing is written unasked

What it costs you

You have to choose it

Nothing is done for you

It cannot plan ahead

It forgets between sessions

THE SAME SHAPE, MORE MACHINERY

Who finds the context

You name the files	Aider, when you add them
A heuristic guesses	Aider, with the repo map
The model asks for it	The tools you meet next

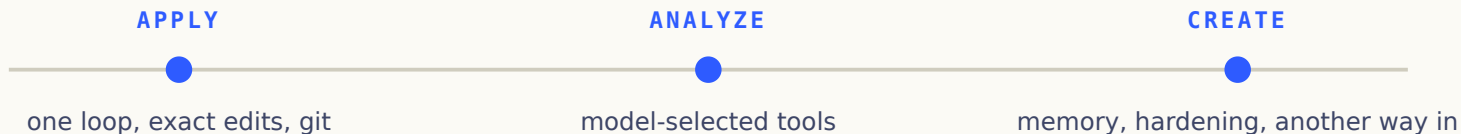
More powerful tools are this shape with more machinery.

What comes next

ONE REPOSITORY, GROWN UNTIL DECEMBER

THROUGH DECEMBER

One repository to December



Nothing is thrown away. Each phase grows the last.

BY DECEMBER

The harness by December

You will have built the kind of tool this lab took apart: a coding agent in the shape of Claude Code or OpenClaw, running against a model you configure, grown from the pair-programmer you start this week.

Today you drive one. In December you will have written one.

Office hours

THE REST OF THE SESSION IS YOURS

Questions?